

## C avancé

### TP1 - Plus loin avec les pointeurs

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

## 1 Pointeurs génériques

L'objectif de cet exercice est de vous faire manipuler des pointeurs génériques.

1. Écrivez une fonction `reverseArrayGeneric` qui inverse les éléments d'un tableau, quelque soit le type et le nombre d'éléments qu'il contient. Le traitement devra se faire « en place » (*i.e.* directement dans le tableau passé en paramètre, sans utiliser de tableau intermédiaire). Inspirez vous de la fonction `memorySwap` vue en cours (mais ne l'utilisez pas directement) !

**N.B.** : Utilisez ChatGPT (ou n'importe quel autre LLM) est contre-productif... L'objectif est de comprendre ce que vous faites. Durant les épreuves notées, vous n'aurez que votre cerveau pour réaliser les exercices...

2. Testez votre fonction avec un tableau d'entiers et un tableau de doubles.
3. Testez maintenant votre fonction avec un tableau de type `Personne` :

---

```
1 typedef struct {
2     char *nom;
3     int age;
4 } Personne;
```

---

## 2 Arithmétique des pointeurs

Ces questions sont reprises de l'exercice 3 du TP3 de « Bases du langage C ». Vous devez y répondre sans utiliser la notation avec crochets sur vos tableaux !

1. Écrivez un programme qui prend en entrée une taille  $N$  et crée un tableau dynamiquement alloué de  $N$  entiers (`malloc`). Vous remplirez ce tableau avec les entiers de 1 à  $N$ . N'oubliez pas de libérer la mémoire à l'aide de la fonction `free`.
2. Écrivez une fonction `displayArray` qui prend en paramètre un pointeur vers un tableau d'entier ainsi que sa taille et affiche son contenu avec un espace entre chaque valeur.
3. Écrivez une fonction `sum` qui calcule et retourne la somme de tous les éléments d'un tableau.

## 3 Tableaux 2D

L'objectif de cet exercice est de manipuler des tableaux 2D via des doubles pointeurs.

1. Écrivez un programme qui prend en entrée un nombre de lignes  $A$  et un nombre de colonnes  $B$ , et crée un tableau dynamiquement alloué de  $A \times B$  entiers (`malloc`). N'oubliez pas de libérer la mémoire à l'aide de la fonction `free`.
2. Remplissez ce tableau de sorte que les cases  $(i, j)$  soient égales à  $(i + 1) \times (j + 1)$ .

3. Écrivez une fonction `displayArray2D` qui affiche le tableau dans la console. Par exemple, pour un tableau de taille  $4 \times 8$ , l’affichage sera (utilisez les tabulations) :

|   |   |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|
| 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  |
| 2 | 4 | 6  | 8  | 10 | 12 | 14 | 16 |
| 3 | 6 | 9  | 12 | 15 | 18 | 21 | 24 |
| 4 | 8 | 12 | 16 | 20 | 24 | 28 | 32 |

4. Reprenez les questions précédentes sans utiliser la notation avec crochet sur vos tableaux!
5. (Bonus) Écrivez une fonction qui réalise la **multiplication de deux matrices**. Une matrice est représentée par un tableau 2D. Cette fonction prend en paramètres les deux matrices à multiplier ainsi que la matrice de résultat. Vous supposerez que les deux matrices sont bien multipliables (à vérifier avant l’appel de la fonction).  
Vous pouvez trouver plus d’information sur le fonctionnement du produit matriciel sur internet. Par exemple : [https://fr.wikipedia.org/wiki/Produit\\_matriciel](https://fr.wikipedia.org/wiki/Produit_matriciel).
6. (Bonus bonus) Réécrivez la multiplication de matrices sans la notation des crochets!